

Task 1

Theoretical Forensics:

The "Heavy Class" Myth

A junior developer on your team argues that we should use structs instead of classes for thousands of 3D game entities because "classes are bloated and take up too much RAM since they store all those functions inside the object."

- Explain mathematically why they are wrong.
- If the functions do not live inside the object in RAM, where exactly in the system's memory layout do they live?
- How does the compiler know which specific object instance the function is modifying?

```
1  #include <iostream>
2  using namespace std;
3
4  class point_1{
5      double x;
6      double y;
7  };
8
9
10 struct point_2{
11     double x ;
12     double y;
13     void print (){
14         cout << x << y << endl;
15     }
16 };
17
18
19 int main (){
20     cout << sizeof (point_1) << " " << sizeof (point_2); // 16 16
21 }
```

A member function is compiled into machine code, which is typically stored in the .text (code) segment. This code is usually stored only once, and all objects can use the same function. Objects store their own data members in memory, such as on

the stack or heap, depending on how they are created. So, using a struct instead of a class does not automatically save memory because both can have the same memory layout and the main difference between a class and a struct in C++ is their default access level. In a class, members are private by default, while in a struct, members are public by default. Both can have data members and member functions, and both can have the same object size.

.This happens because of the this pointer. It is a pointer that refers to the current object that called the member function. The this pointer is implicitly available inside every non-static member function. It allows the function to access the data members of the correct object, so there is no confusion between different objects.

The Memory Map

BSS → (int global_player_count = 0)

Stack → (*boss , int temp_score)

Code/Text → (void calculatephysics())

Heap → (new Enemy ())

The cliffhanger

We need automatic memory cleanup when an object goes out of scope

Code Review & Bug Hunting

1-The memory is allocated before checking if the size is valid. If the size is zero, the function returns without calling delete[], which may cause a memory leak.

```
void processClientRequest(int dataSize) {  
    if (dataSize <= 0) {  
        std::cout << "Invalid request size." <<std:: endl;  
        return;  
    }  
  
    // Process the data buffer  
    int* dataBuffer = new int[dataSize];  
    delete[] dataBuffer;  
}
```

2- The function (getTemperaturePointer()) returns a pointer to the private data member and this allows the user to change the private data directly .

```
double getTemperature() const {  
    return core_temperature;  
}
```

Lap1

```
#include <iostream>  
using namespace std;  
// Lap 1  
class DynamicScoreList{  
public:  
int *allocateScores(int size){  
    int *Scores = new int [size];  
    return Scores;  
}  
  
void freeScores(int* ptr){  
    delete [] ptr;  
}  
};  
int main (){  
    DynamicScoreList Score;  
    for ( int i = 0; i < 10000; ++i){  
        int *score = Score.allocateScores(100);  
    }  
}
```

Lap2

```
#include <iostream>
using namespace std;
// Lap 2
class Player{
private:
int health = 100;
public:
void takeDamage ()
{
    health -= 50;
}
int Gethealth(){
    return health;
}
};

void applyDamageValue(Player p){
    p.takeDamage();
}
void applyDamagePointer(Player * p){
    if (p != nullptr)
        p->takeDamage();
}

void applyDamageReference(Player &p){
    p.takeDamage();
}
```

```
int main (){
    Player p1 ;
    applyDamageValue(p1);
    cout << "By Value: " << p1.Gethealth(); // 100
    cout << endl;
    applyDamagePointer(&p1);
    cout << "By Pointer : " << p1.Gethealth(); // 50
    cout << endl;
    applyDamageReference(p1);
    cout << "By Reference : "<< p1.Gethealth(); // 0
    cout << endl;
}
```